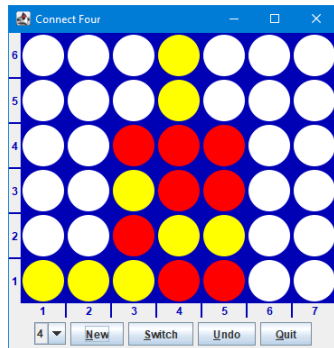


Aufgabe 3

Vier gewinnt graphische Bedienoberfläche

- ▶ Graphisches Userinterface zu Aufgabe 2
- ▶ JAVA Swing
- ▶ Model-View-Controller Systemarchitektur
 - ▶ Model schon in Aufgabe 2 realisiert
 - ▶ Interface Board weiterhin verbindlich
 - ▶ Änderungen in Klassen (eigentlich) nicht notwendig
 - ▶ Ggf. Synchronisierung
- ▶ Anregung
- ▶ Ziele
 - ▶ Arbeiten mit Swing-Komponenten wie Menüs, Listeners, ...
 - ▶ Sinnvolle Benutzerführung
 - ▶ Wiederverwenden von vorhandener Implementierung



Aufgabenstellung

- ▶ Geforderte Funktionalität
 - ▶ Zur Zugeingabe ist das Spielfeld mit der Maus klickbar
 - ▶ Ganze Spalte klickbar
 - ▶ Maschinenzug kommt bei Bedarf automatisch
 - ▶ Neubeginn eines Spiels
 - ▶ Einstellmöglichkeit für Schwierigkeitslevel
 - ▶ Wechsel des Eröffners
 - ▶ Startet neues Spiel
 - ▶ Benachrichtigung des Spielers
 - ▶ Mensch gewinnt
 - ▶ Maschine gewinnt
 - ▶ Unentschieden
 - ▶ Illegaler Zug
 - ▶ In einer Siegsituation wird der/ein Zeuge visualisiert
 - ▶ Beenden des Programms

Aufgabenstellung. . .

- ▶ Um die GUI immer bedienbar zu halten wird der Maschinenzug in einem eigenen Thread berechnet
 - ▶ Bei höherem Skill-Level (Look-Ahead praktisch durchführbar bis zu 7) dauert Maschinenzug deutlich an
 - ▶ EventDispatcher kann weiterhin auf Benutzereingaben reagieren
- ▶ Während Maschinenzug
 - ▶ Manuelle Zugeingabe ist gesperrt (Nutzer ist noch nicht dran)
 - ▶ System reagiert auf die anderen Kommandos New, Switch, Quit, Leveländerung zeitnah
 - ▶ Warten auf das Ende eines Maschinenzugs ist dabei explizit nicht notwendig
 - ▶ Da die Zugberechnung vom Model nicht auf regulären vorzeitigen Stop ausgelegt ist, soll der Berechnungsthread via `interrupt()` beendet werden
 - ▶ In (den Hilfsmethoden von) `machineMove()` gelegentlich auf `Thread.interrupted()` prüfen und wenn ja `InterruptedException` werfen
 - ▶ Die Exception in der `run`-Methode des Berechnungsthreads fangen und diese beenden
 - ▶ Leveländerungen brauchen/sollen erst beim nächsten Maschinenzug berücksichtigt werden

Model-View-Controller Architektur

- ▶ Komponenten

- Model** Repräsentation des aktuellen Zustands und automatisches Ziehen

- View** Benutzeroberfläche

- ▶ Panels, Knöpfe, Menüs, ...

- Controller** Sammlung der verschiedenen (Action-, Mouse-)Listener der GUI

- ▶ Vereinfachter Ablauf

1. View (und damit Controller als innere Klassen) hat Referenz auf das Model
2. Controller ruft Änderungsmethoden des Models auf und veranlasst ein Neuzeichnen (geeigneter) Komponenten
3. View aktualisiert sich nach Statusabfrage beim Model

- ▶ Alternativ Observer-Pattern

- ▶ Board bzw. Game sind schlecht geeignet als Observable zu fungieren
 - ▶ Board sollte nicht verändert werden
 - ▶ Objekte sind Cloneable → das Observable-Objekt wird ausgetauscht
 - ▶ Am einfachsten per Zwischenschicht/Wrapper DisplayData für Game
 - ▶ Dieser Wrapper implementiert dann Observable und macht i. W. nur Delegation an (das aktuelle) Game
 - ▶ Die Delegation/Kommunikation darf natürlich auch das Interface Board nicht verletzen/umgehen

Implementierung

- ▶ Spielfeldgröße als $\# \text{Zeilen} \times \# \text{Spalten}$ nicht hartcodiert
 - ▶ Verwendung der Konstanten aus Board
- ▶ Vorschlag
 - ▶ Spielfeld an sich als eigenes JPanel
 - ▶ Benutzt GridLayout
 - ▶ Jede Stelle im Grid (Slot) als eigenes JPanel
 - ▶ Stellt sich selber mittels `paintComponent()`-Methode dar
 - ▶ Erst Hintergrund richtig zeichnen
 - ▶ Sonst Schmutzeffekte bei Neuzeichnen
 - ▶ Falls Änderung auftritt `repaint()`
- ▶ Referenzimplementierung hat ca. 800 Zeilen Code mit voller Kommentierung
 - ▶ Zusätzlich zum Code aus Aufgabe 2

Termine

- ▶ Bekanntgabe Montag, 10. Januar, 8:00 Uhr
- ▶ Empfohlene Einreichung der Lösung Freitag 4. Februar, 10:00 Uhr
- ▶ Letzte Möglichkeit Lösung abzugeben, Freitag 11. Februar, 10:00 Uhr
- ▶ Fragen jetzt oder im entsprechenden Forum in Stud.IP
- ▶ Bei technischen Problemen: bachmaier@fim.uni-passau.de